

1 · Estructura de una consulta

Toda consulta empieza con `MATCH` y termina con `RETURN`. Entre medio van otros statements; los comentarios usan `//`.

```
// edad y nombre de quienes conocen a John
MATCH (?x :Person)-[:Knows]->(John)
WHERE ?x.age >= 60 AND ?x.age <= 70
ORDER BY ?x.name DESC, ?x.age ASC
RETURN ?x.age, ?x.name
LIMIT 1000
```

Orden general: `MATCH` / `WHERE` / `LET` / `CALL` (uno o más) → `GROUP BY` → `HAVING` → `ORDER BY` → `RETURN`.

2 · Objetos y variables

Un nodo puede referirse a un objeto fijo (por id) o ligarse a una variable `?x`.

Tipo	Ejemplo
NamedNode	(Juan_Perez)
Nodo anónimo	(_a123)
Arista (por id)	(_e123)
String	"texto"
Booleano	true false
Entero / Float	123 3.14
Variable	?x

3 · Patrón de nodo

```
() // cualquier objeto
(Juan_Perez) // objeto fijo por id
(?x) // variable
(:Diputado) // por etiqueta
(:Diputado :Persona) // varias etiquetas
(?x :Diputado)
(?x {edad: 42}) // por propiedad
(?x :Diputado {region: "Maule"})
```

Un nodo admite *varias* etiquetas; las propiedades van al final.

4 · Patrón de arista (dirigida)

Las aristas siempre llevan dirección y a lo sumo **un** tipo.

```
(?x)-[?e]->(?y) // variable de arista
(?x)-[_e123]->(?y) // arista fija por id
(?x)-[:Vota]->(?y) // tipo fijo
(?x)-[?e :Vota]->(?y) // variable + tipo
(?x)-[:?t]->(?y) // variable de tipo
(?x)-[{peso: 3}]->(?y) // con propiedades
(?y)<-[:Vota]-(?z) // sentido inverso
```

5 · Caminos (RPQ)

Conectan dos nodos por una *expresión regular de caminos* sobre tipos. Delimitadores: `= [ ... ] =>` (derecha) y `<= [ ... ] =` (izquierda).

<code>^t</code>	tipo <code>t</code> en sentido inverso
<code>t*</code>	0 o más repeticiones
<code>t+</code>	1 o más repeticiones
<code>t?</code>	0 o 1 repetición
<code>t1/t2</code>	secuencia (t1 luego t2)
<code>t1 t2</code>	alternativa (t1 o t2)
<code>{m,n}</code>	entre m y n repeticiones

```
(?x)=[ :Conoce+ ]=>( ?y)
(Foo)<=[ ALL ?p :Conoce+ ]=( ?z)
(Foo)=[ ANY ?p :Vota+ :De? ]=>(Bar)
```

Semántica opcional antes del tipo: `ANY` (def.) o `ALL`; modos `SHORTEST`, `SIMPLE`, `ACYCLIC`, `TRAILS`, `WALKS`. Variable de camino `?p` opcional.

6 · Filtros dentro de { }

Dentro de las llaves de un nodo/arista se filtra directamente:

<code>{k: v}</code>	igualdad exacta
<code>{k op v}</code>	<code>==</code> <code>!=</code> <code><</code> <code>></code> <code><=</code> <code>>=</code>
<code>{k IS [NOT] T}</code>	T = NULL, STRING, BOOL, INTEGER, FLOAT
<code>{k: date("...")}</code>	valor con tipo de dato

```
(?p :Diputado {edad > 30})
(?p :Diputado {region: "Maule",
               activo IS NOT NULL})
```

7 · WHERE y operadores

`WHERE` filtra con una expresión que debe evaluar a verdadero.

```
MATCH (?x :Diputado)
WHERE ?x.edad >= 30 AND ?x.edad <= 80
AND ?x.region != "Maule"
RETURN ?x
```

Lógicos	<code>AND</code> <code>OR</code> <code>NOT</code>
Comparación	<code>==</code> <code>!=</code> <code><</code> <code>></code> <code><=</code> <code>>=</code>
Aritméticos	<code>+</code> <code>-</code> <code>*</code> <code>/</code> <code>%</code>
Tipo	<code>?x.k IS NOT NULL</code>
Igualdad de objeto	<code>?x == (Foo)</code>

Ojo: la igualdad es `==` (no `=`). El acceso a propiedad es `?x.clave`.

8 · Filtros con expresiones regulares

`REGEX(texto, "patrón", "flags")` es verdadero si *texto* calza con la regex. El 3.^{er} argumento (flags) es opcional.

```
MATCH (?p :Proyecto)
WHERE REGEX(?p.titulo, "educaci[oó]n", "i")
RETURN ?p
```

- `"i"` = *case insensitive* (ignora mayús/minús).
- Patrón sobre la propiedad: `"^Ley"` (empieza), `"reforma$"` (termina), `"salud|salario"` (alternativa).

9 · Statements principales

LET (variables derivadas)

```
MATCH (?a :Diputado)
LET ?decada = ?a.edad / 10
RETURN ?a.nombre, ?decada
```

RETURN: `DISTINCT`, `*`, *alias*

```
RETURN *
RETURN DISTINCT ?x
RETURN ?x.nombre AS ?nombre, ?x.edad
```

ORDER BY · **LIMIT** · **OFFSET**

```
MATCH (?a :Diputado)
ORDER BY ?a.edad DESC
RETURN ?a.nombre, ?a.edad
OFFSET 3
LIMIT 10
```

OPTIONAL (unión opcional, tipo left-join)

```
MATCH (?x :Diputado)
OPTIONAL { (?x)-[:Preside]->( ?c) }
RETURN ?x.nombre, ?c
```

10 · Funciones y agregación

Agregación: **GROUP BY** / **HAVING**

```
MATCH (?a :Diputado)
GROUP BY ?a.region
HAVING COUNT(*) > 5
RETURN ?a.region, COUNT(*) AS ?n
```

Agregados: COUNT (con DISTINCT), SUM AVG MIN MAX .

Funciones útiles

REGEX (s,p,f?)	coincidencia regex
EDIT_DISTANCE (a,b)	distancia de edición
STR (x)	a string
LABELS (x)	etiquetas del nodo
TYPE (e)	tipo de la arista
PROPERTIES (x)	propiedades
NORMALIZE (v)	normaliza vector

11 · Similitud de vectores

Los *embeddings* se guardan como tensores 1-D en una propiedad (p.ej. `embedding`). Métricas: `COSINE_DISTANCE` , `COSINE_SIMILARITY` , `EUCLIDEAN_DISTANCE` , `MANHATTAN_DISTANCE` .

Fuerza bruta (exacto)

```
LET ?fijo = nodoConsulta
MATCH (?d :Intervencion)
LET ?dist = COSINE_DISTANCE(
    ?fijo.embedding, ?d.embedding)
ORDER BY ?dist
RETURN ?d, ?dist
LIMIT 100
```

Menor distancia coseno = más similar.

Aproximado con índice HNSW (rápido a gran escala)

```
CALL HNSW_TOP_K("mi_indice",
    ?fijo.embedding, 100, 1000)
YIELD ?object AS ?similar, ?dist
RETURN ?similar, ?dist
```

Argumentos: `(indice, vector, k, ef)` . El índice se crea aparte:

```
CREATE HNSW INDEX "mi_indice" WITH {
    "property"= "value",
    "dimension"= 768,
    "maxCandidates" = 16,
    "maxEdges" = 8,
    "metric"= "cosineDistance"
}
```

12 · Recetas típicas para GraphRAG

Top-k denso sobre intervenciones

```
LET ?q = consultaEmbedding
MATCH (?i :Intervencion)
LET ?s = COSINE_DISTANCE(?q, ?i.embedding)
ORDER BY ?s
RETURN ?i.texto, ?s
LIMIT 10
```

Filtrar por metadato y luego rankear

```
MATCH (?i :Intervencion)
-[:De]-(?a :Diputado)
WHERE ?a.partido == "PartidoX"
LET ?s = COSINE_DISTANCE(?q, ?i.embedding)
ORDER BY ?s
RETURN ?i.texto, ?a.nombre, ?s
LIMIT 10
```

Expansión por grafo (vecinos del resultado)

```
MATCH (?i :Intervencion)
-[:Menciona]-(?p :Proyecto)
WHERE REGEX(?p.titulo, "pension", "i")
RETURN DISTINCT ?p.titulo
```